Module 10: Linux Exploitation and Privilege Escalation

```
# Exercise 02: Searching for Files with SUID/SGID Permissions
```

Scenario

SUID/SGID Permission Hunting & Privilege Escalation Reconnaissance audits special Linux file system permission bits that allow binaries to execute with the privileges of the file owner (SUID, `u+s`, bit `4000`) or group owner (SGID, `g+s`, bit `2000`). When misconfigured on administrative utilities or custom binaries (e.g., `pkexec`, `find`, `nmap`, `vim`, `env`), non-privileged users can exploit these binaries to execute arbitrary commands as `root`.

Key technical concepts in SUID/SGID binary auditing:
1. **Target SSH Enumeration**: Connecting from Parrot Security to target host **192.168.0.51** via SSH (**ubuntu** / **toor**) and verifying active identity (**whoami; id**) and kernel release (**uname -r; cat /etc/issue**).
2. **SUID Binary Search (`-perm -4000`)**: Using the Linux `find` utility with permission mask `-4000` redirecting stderr errors (`2>/dev/null`) to locate all binaries running with elevated owner privileges:
find / -type f -perm -4000 2>/dev/null
3. **Detailed Root SUID File Listing (`-user root -ls`)**: Filtering SUID binaries owned specifically by `root` with file size, inode, timestamp, and path details:
find / -type f -perm -4000 -user root -ls 2>/dev/null
4. **SGID Binary Search (`-perm -2000`)**: Searching for binaries running with elevated group owner privileges:
find / -type f -perm -2000 2>/dev/null

In this exercise, we log into **Parrot Security** (**pentester** / **toor**), SSH into target **Ubuntu 22** server **192.168.0.51**, inspect user/kernel environment, execute SUID search commands (`find / -type f -perm -4000 2>/dev/null`), filter root-owned SUID binaries with long-listing output (`-ls`), and execute SGID search commands (`find / -type f -perm -2000 2>/dev/null`).

Objective

The objective of this lab is to help students learn how to:
- Authenticate to Parrot Security VM (**pentester** / **toor**)
- SSH into target Ubuntu 22 VM: **ssh ubuntu@192.168.0.51** (Password: **toor**)
- Audit system identity and OS build: **whoami; id** and **uname -r; cat /etc/issue**
- Perform manual search for SUID binaries: **find / -type f -perm -4000 2>/dev/null**
- List detailed root-owned SUID file permissions: **find / -type f -perm -4000 -user root -ls 2>/dev/null**
- Perform manual search for SGID binaries: **find / -type f -perm -2000 2>/dev/null**
- Differentiate default system SUID binaries from custom exploitable privilege escalation vectors

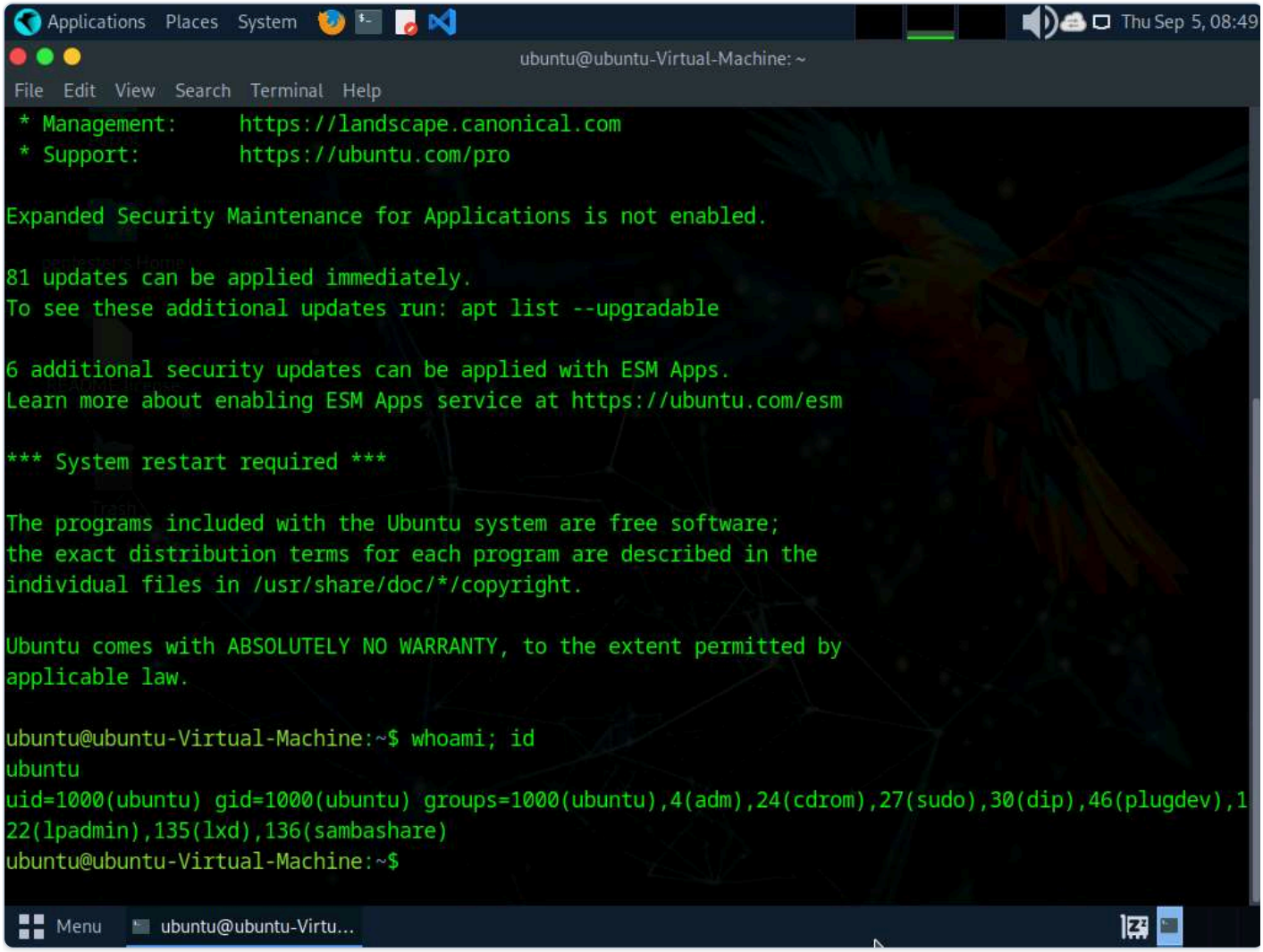
---

Task 1: Connect to Target Server via SSH, Audit System Properties, and Search for SUID & SGID Binaries

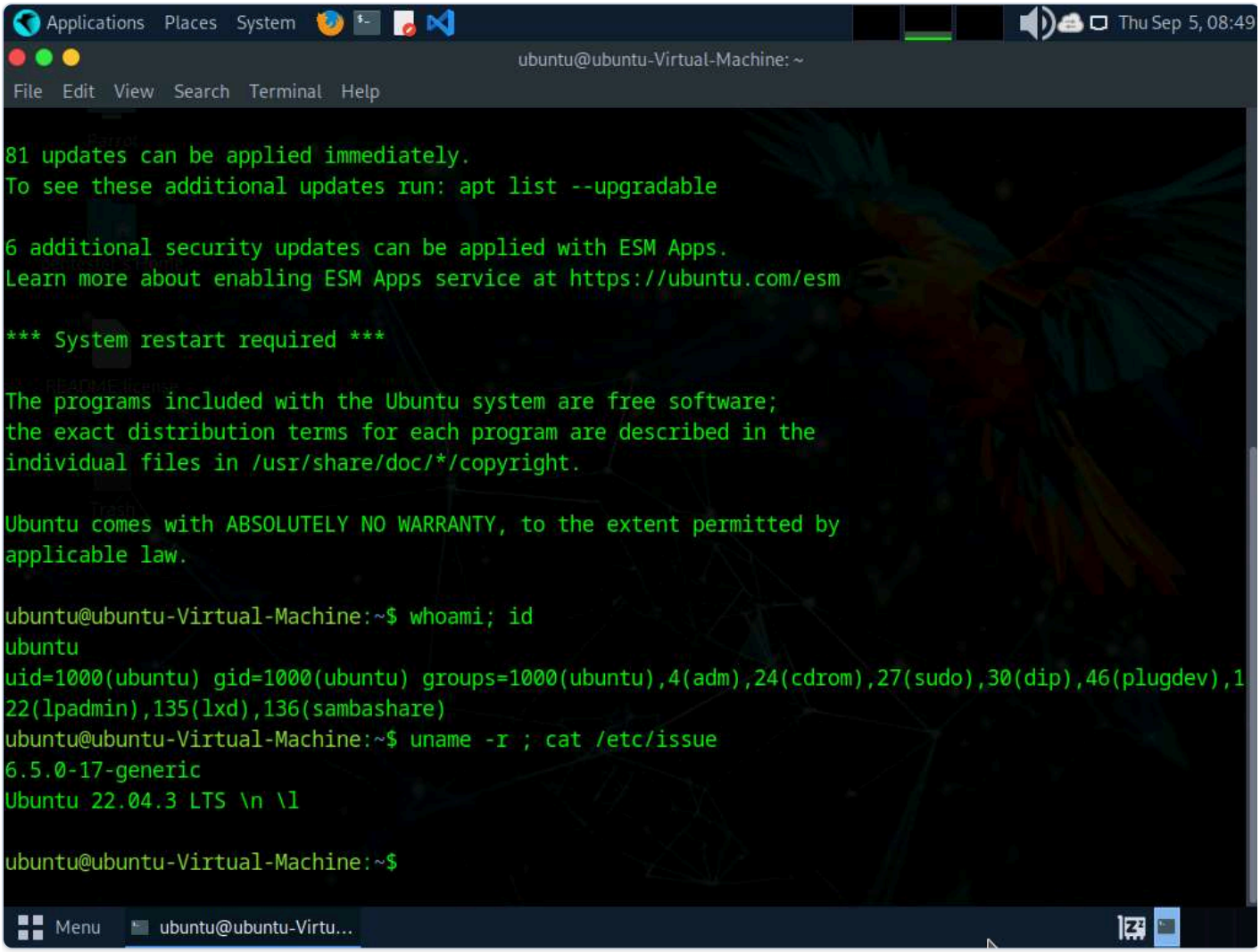
In this task, we establish SSH access to `192.168.0.51`, verify identity and kernel version, search for SUID binaries (`-perm -4000`), list detailed root SUID files (`-user root -ls`), and search for SGID binaries (`-perm -2000`).

Lab Tasks

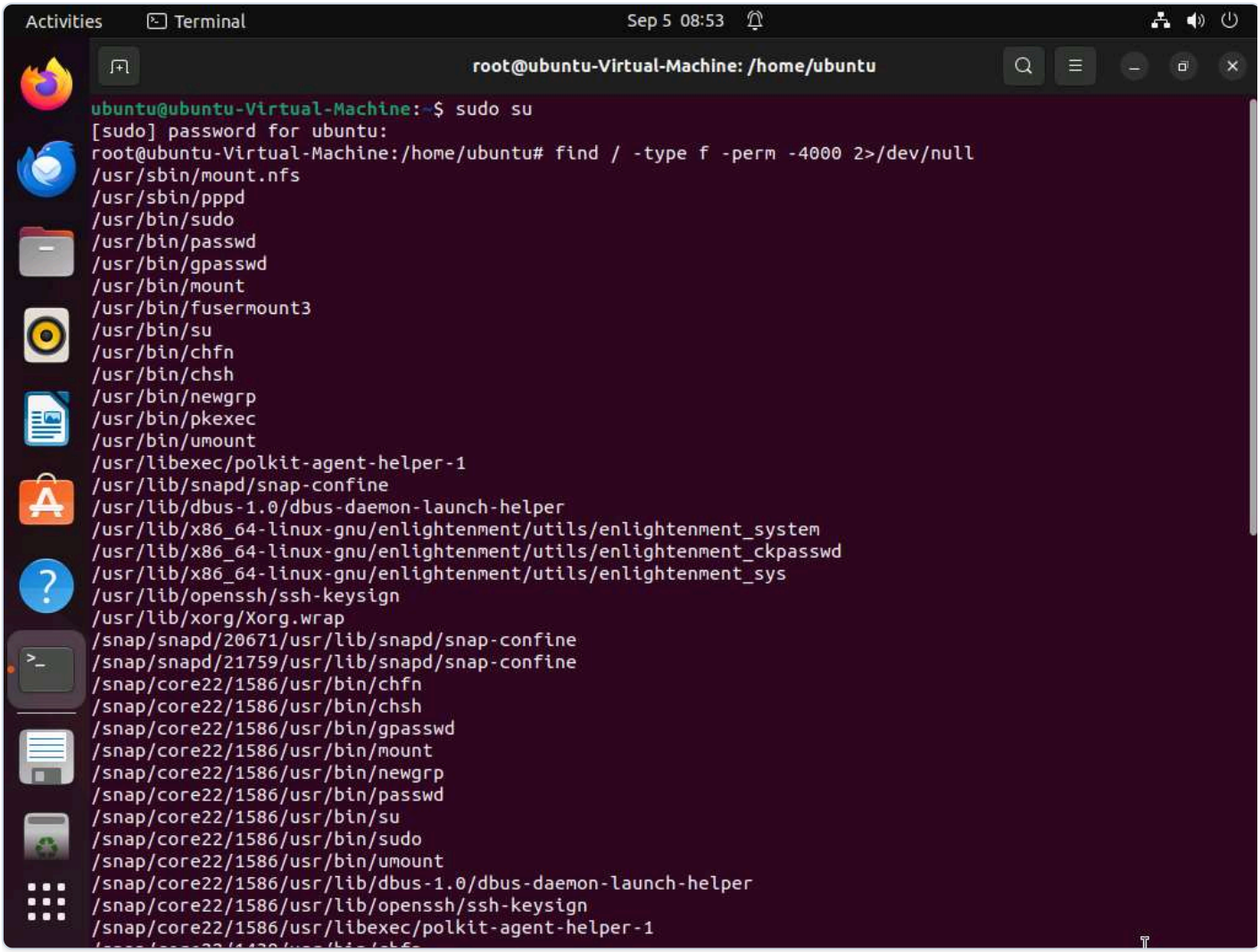
1. Log into **Parrot Security** VM using credentials (**pentester** / **toor**). Open terminal, obtain root privileges (`sudo su`), and connect via SSH to target Ubuntu host **192.168.0.51**:
   ssh ubuntu@192.168.0.51
   Authenticate with password **toor**. Check active user identity:
   whoami; id



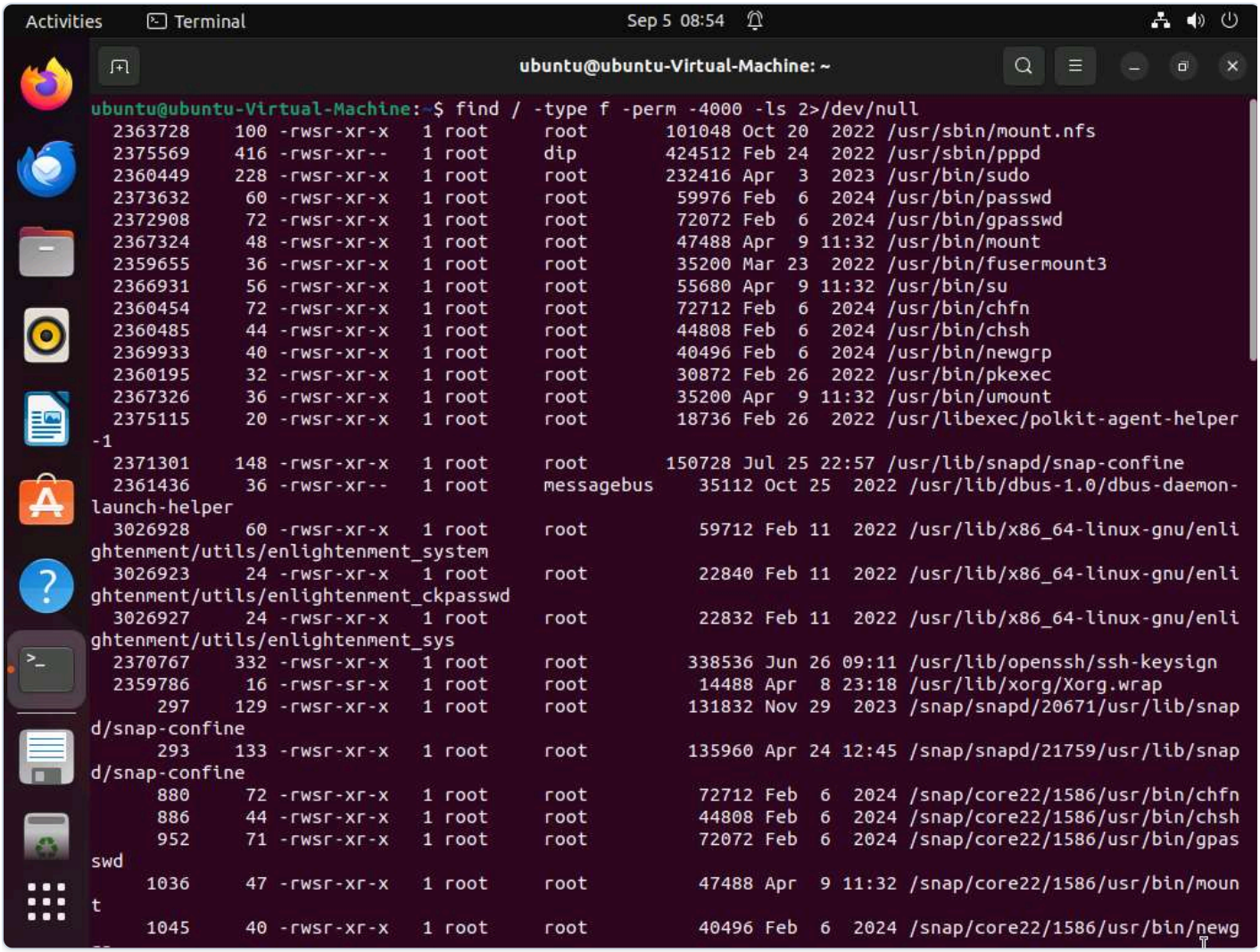
1. Inspect Linux kernel release and operating system issue version:
   uname -r ; cat /etc/issue



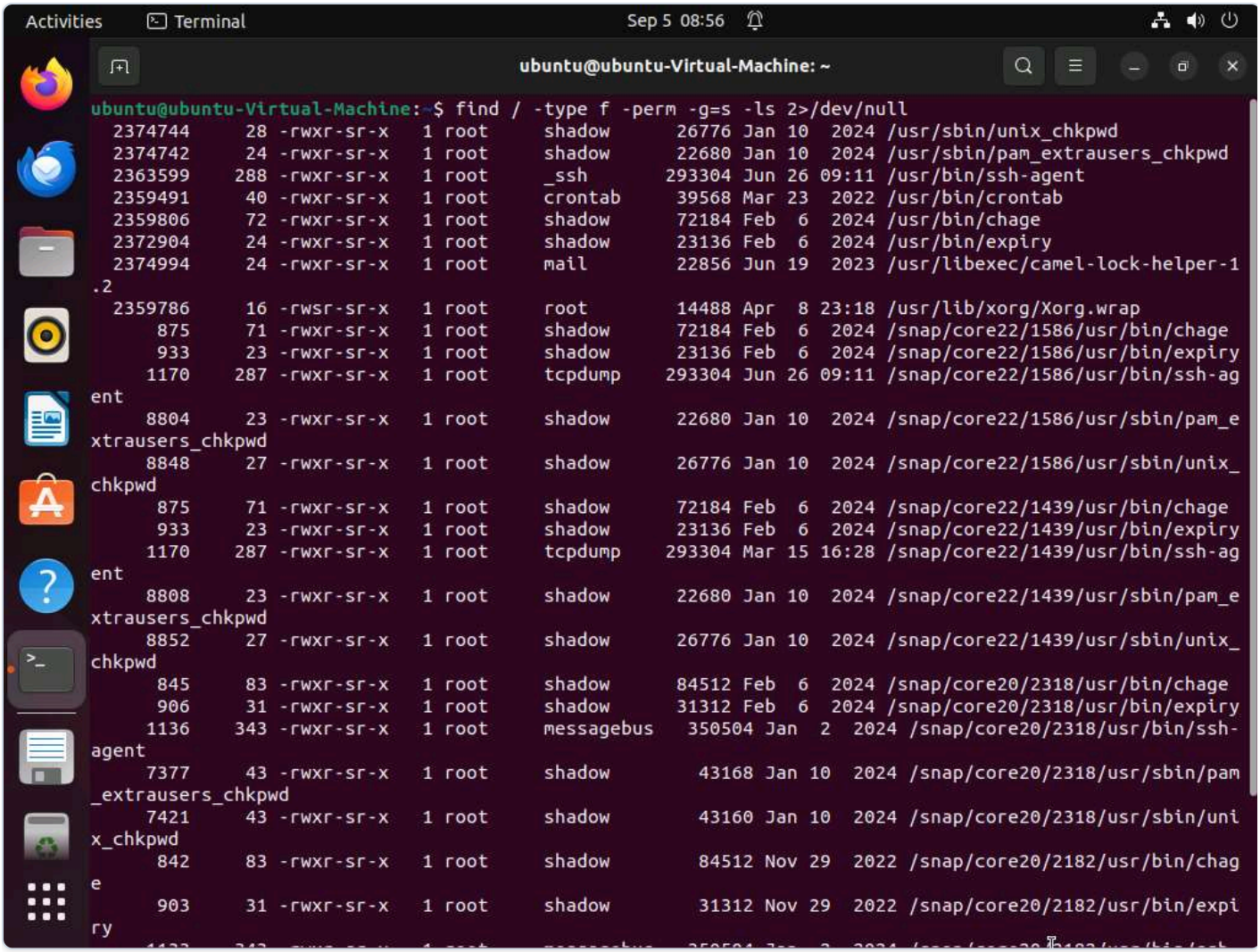
1. Switch to target **Ubuntu** machine (or terminal session) and obtain root privileges (`sudo su`). Search for all SUID binaries (`-perm -4000`) suppressing standard error messages:
   find / -type f -perm -4000 2>/dev/null



1. Perform detailed long-format listing (`-ls`) of root-owned SUID binaries to analyze file ownership and permissions:
   find / -type f -perm -4000 -user root -ls 2>/dev/null



1. Search for SGID binaries (`-perm -2000`) suppressing standard error messages:
   find / -type f -perm -2000 2>/dev/null



This concludes the lab exercise.